API & Integration Test Case Design

Hands-On Student Activity Sheet & Design Template

Student Name:	Janallan D. Pandiin
Date / Class Section:	September 15, 2026 - IT 41
Course / Module:	SIA 2 - IT 42 (Integration & Software Architecture)
Final Score / Grade:	____ / 100 Points

Overview & Educational Objectives

This workbook serves as an active-learning tool to bridge the gap between high-level testing theory and concrete, technical execution. Unit testing ensures that an isolated function or method works correctly using test doubles. However, as distributed, service-oriented architectures dominate the software landscape, the majority of critical defects—such as connection timeouts, schema mismatches, serialization failures, and authorization gaps—manifest at the communication boundaries between systems.

By completing this workbook, you will practice designing robust integration and API contract test cases aligned with international standard frameworks like ISO/IEC/IEEE 29119-3, OWASP API Security, GDPR, and WCAG 2.2. This workbook is fully harmonized with your grading sheet; the table structures are designed to match the Technical Targets, Payloads, Assertions, and Gate Criteria that will be evaluated on your final submission.

Section 1: The Schema Validation & Glue Defect Challenge

Background Scenario: Your team is integrating a legacy Shipping Discount calculator database with a modern Order checkout API. During unit testing, the developers used mocks and stubs, assuming the database would return the inventory status field as an integer (e.g., 12). The unit tests passed with 100% success. However, in production, the legacy database returned values as strings (e.g., '12'). This schema mismatch caused a silent system-wide calculation crash that halted checkout.

Exercise 1.1: The Schema Validation Test Design (JSON Schema Mismatch)

Task: Design an API Contract & Schema Validation test case using the table template below. Detail how you will programmatically detect the 'integer-to-string' type mismatch on the response payload at the API interface level before the payload is allowed to propagate to downstream services.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-1.1.1	POST /api/checkout - response schema field `inventory_status` (contract-defined type: integer)	Mock legacy DB response: {"item_id":"SKU-2231","inventory_status":"12"} (string instead of integer)	1) Intercept the response at the API gateway/contract layer before it reaches business logic. 2) Validate the payload against the published JSON Schema (type: integer) using a validator (e.g., AJV, OpenAPI schema validation). 3) Assert typeof(inventory_status) === "number". 4) On mismatch, reject the payload and raise a structured contract-violation error instead of passing it downstream. 5) Confirm the checkout service degrades gracefully (e.g., 502 Bad Gateway with error code) rather than crashing.	PASS: Validator detects the type mismatch and blocks propagation; checkout returns a controlled error. FAIL: String value passes validation, reaches the calculation service, and causes a crash or silent miscalculation.

Exercise 1.2: Catching 'Glue' Defects

Task: Design three specific test cases to catch other common integration 'glue' defects: 1) Connection Timeouts (enforcing a strict 5-second failure limit on external gateway queries), 2) Auth Scope Mismatches, and 3) JSON Serialization errors.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-1.2.1	External Shipping Gateway API - POST /gateway/rates	Service-virtualized gateway configured to delay response by 7 seconds (exceeds 5s SLA)	1) Send a rate-quote request to the gateway. 2) Start a client-side timer at request dispatch. 3) Assert the HTTP client aborts the call at 5.000s (±100ms tolerance). 4) Assert a circuit breaker/fallback response (e.g., cached rate or 'unavailable' message) is returned to the caller. 5) Verify no thread/connection is left hanging after timeout.	PASS: Request is aborted at 5s and a graceful fallback is returned. FAIL: Request hangs beyond 5s, blocks the checkout thread, or the app becomes unresponsive.
TC-1.2.2	PUT /api/inventory/update - requires OAuth scope `inventory:write`	Valid JWT access token issued with only the `inventory:read` scope, used to call the write endpoint	1) Authenticate and obtain a token scoped only for read access. 2) Send a PUT request to the write endpoint using that token. 3) Assert the API returns HTTP 403 Forbidden (not 200 or 500).	PASS: 403 returned, no data mutation occurs, error message is generic/safe. FAIL: Request succeeds (200), a partial write occurs, or the error leaks internal details.

			4) Query the database directly to confirm no state change occurred. 5) Confirm the error response does not leak internal scope names, stack traces, or implementation details.	
TC-1.2.3	GET /api/orders/{id} - JSON response serializer	Order record containing a malformed/circular nested object and a null required date field	1) Trigger serialization of the problematic order record. 2) Assert the serializer catches the failure internally (try/catch around the serialization boundary). 3) Assert the client still receives a syntactically valid JSON body (not truncated or malformed). 4) Assert the error body is structured (e.g.,{"error":"SERIALIZATION_FAILED","code":"ERR_500"}) with no raw stack trace. 5) Confirm the failure is logged server-side with enough context to debug.	PASS: Client receives valid JSON with a structured error; failure is logged. FAIL: Client receives invalid JSON, a raw exception/stack trace, or the connection is dropped.

Section 2: The API Security & Regulatory Compliance Drill

Background Scenario: A distributed health insurance platform allows patients to retrieve their medical claims via an API endpoint `/orders/{order_id}`. This architecture faces severe risks from OWASP API Risk #1: Broken Object Level Authorization (BOLA), where an attacker replaces their own ID with another patient's ID to download private files. Additionally, GDPR and HIPAA laws strictly prohibit the use of cloned production data containing Protected Health Information (PHI) in test environments.

Exercise 2.1: Designing a BOLA Penetration Test

Task: Design a step-by-step penetration test case that verifies the system actively blocks authorization spoofing. Show how an API request using a valid, active session token belonging to User A will be denied and return a secure, structured HTTP error state when it attempts to query User B's resource ID.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-2.1.1	GET /orders/{order_id} - Broken Object Level Authorization (OWASP API1)	Valid, active session token for User A (order_id=1001); attacker modifies the path to order_id=1002 (belongs to User B)	1) Authenticate as User A and capture the valid session token. 2) Replay the request against User A's own order_id to confirm normal 200 success (control case). 3) Replay the identical token against User B's order_id (1002). 4) Assert the response is HTTP 403 Forbidden or 404 Not Found - never 200. 5) Assert the response body contains no fragment of User B's claim/PHI data. 6) Confirm the attempt is written to a security audit log with user, resource, and timestamp.	PASS: Cross-ID access is blocked with a structured, non-leaking error and the attempt is logged. FAIL: User B's order/claims data is returned in the response body under User A's token.

Exercise 2.2: Compliance-Driven Test Data Management (TDM)

Task: Draft a structured plan to programmatically generate realistic synthetic data for testing. Your schema must outline how sensitive personal data fields are randomized or masked to respect GDPR data minimization and HIPAA auditing without losing the structured format necessary for execution.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-2.2.1	Test Data Management (TDM) pipeline / synthetic data generation service for the patient claims schema	Field-level generation spec, e.g.: name → Faker.name(), SSN → format-preserving fake (###-##-####, non-issued range), DOB → randomized within valid age bounds, diagnosis_code → sampled from valid ICD-10 code list (not tied to a real patient)	1) Define a data-masking schema mapping every PHI field to a synthetic generator that preserves data type and format. 2) Generate a batch of synthetic patient records. 3) Run a PHI-detection scan (regex + known real-value lookup) confirming zero matches to real production values. 4) Assert referential integrity is preserved (e.g., a synthetic patient_id still correctly links to synthetic claims records). 5) Load the synthetic dataset into the test environment and confirm the API executes normally against it.	PASS: 0% real PHI detected in generated data; referential integrity and format are intact; tests execute successfully. FAIL: Any real PII/PHI value is found, or the synthetic data breaks schema/foreign-key constraints.

Section 3: The Latency & User Experience Integration Challenge

Background Scenario: An application utilizes an external third-party ERP to process invoice submissions. This connection typically takes between 4 and 6 seconds. If the frontend UI does not handle this background latency, users assume the system has crashed and click the 'Submit' button repeatedly. This triggers duplicate database transactions and unhandled backend errors. Conversely, an 'Optimistic UI' (which assumes success instantly) can leave users stranded if the background integration call ultimately fails.

Exercise 3.1: UI State Validation Under Latency

Task: Design test cases that validate the user interface handles latencies correctly using: 1) Skeleton Screens (perceived performance), 2) Progress Bars (predictable durations), and 3) Asynchronous background processing (heavy reporting tasks).

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-3.1.1	Invoice submission frontend component calling POST /invoices (ERP integration, 4-6s latency)	Simulated ERP response delay of 5 seconds via mocked network layer	1) Trigger invoice submission. 2) Assert a skeleton placeholder renders within 200ms of the click (perceived responsiveness). 3) Hold the mocked response for the full 5s delay and confirm the skeleton persists without layout shift. 4) Assert the skeleton is replaced by real content only once the response actually arrives. 5) Confirm the Submit button is disabled during this window to prevent duplicate submissions.	PASS: Skeleton displays immediately, persists correctly, and no duplicate submission is possible. FAIL: Blank/frozen screen appears, or the button remains clickable, allowing duplicate requests.
TC-3.1.2	Invoice sync progress indicator (WebSocket/polling channel reporting ERP	Backend emits progress events at 25%, 50%, 75%, 100% over a 6-	1) Start the invoice sync action. 2) Assert the progress	PASS: Progress bar accurately and monotonically tracks backend state and

	job progress)	second window	bar updates to match each server-reported percentage within 500ms of the event. 3) Assert the bar never regresses (no backward jumps). 4) Assert 100% is displayed only after the server sends the final completion event, not optimistically. 5) Verify the UI recovers gracefully if a progress event is dropped (e.g., resumes from last known state).	completes only on true completion. FAIL: Bar stalls, jumps inconsistently with backend state, or shows 100% before the job actually finishes.
TC-3.1.3	Heavy report generation - POST /reports/generate (async job pattern)	Trigger a report job; poll GET /reports/status/{job_id}	1) Call POST /reports/generate and assert an immediate HTTP 202 Accepted with a job_id (not a blocking wait). 2) Assert the UI immediately shows a 'Processing' state and remains interactive (user can navigate elsewhere). 3) Poll or subscribe for job status until completion. 4) Navigate away from the page and return; assert the job status/progress is still correctly reflected (not lost). 5) Assert a completion notification	PASS: UI stays responsive throughout, job status survives navigation, and completion is reliably signaled. FAIL: UI blocks while waiting, or job status/progress is lost when the user navigates away and returns.

			(toast/email/webhook) fires once the job finishes.	

Exercise 3.2: Optimistic UI Failure Rollback paths

Task: Design a test case validating that if an Optimistic UI action fails on the backend, the frontend gracefully rolls back to its prior state and translates technical errors into blame-free, human-readable instructions without exposing raw JSON exceptions or server stack traces.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-3.2.1	'Mark Invoice Paid' optimistic UI action - PATCH /invoices/{id}	Client optimistically renders 'Paid' state immediately; mocked backend returns HTTP 500 after a 2s delay	1) Click 'Mark as Paid' and assert the UI immediately (optimistically) shows 'Paid'. 2) Allow the mocked backend to return a 500 error. 3) Assert the UI reverts to the prior 'Unpaid' state within 1 second of receiving the error. 4) Assert a plain-language, blame-free message is shown (e.g., "We couldn't update this invoice. Please try again.") - never the raw JSON error or stack trace. 5) Re-fetch the invoice from the server and confirm client state now matches server state exactly (no permanent desync).	PASS: UI rolls back correctly, shows a friendly message, and stays in sync with the server. FAIL: UI remains stuck showing 'Paid' despite the failure, or exposes raw technical error output to the user.

Section 4: The Cross-Platform Journey, Accessibility, & SSO Audit

Background Scenario: Users log into a unified Employee Portal, click on a link that dynamically embeds a third-party HR Benefits platform inside an iFrame, execute a task, and then return to the main Portal. If the single sign-on (SSO) session handshake or the back-button path is misconfigured, users are forced to re-enter credentials or are redirected to dead ends. Furthermore, any accessibility violations within the embedded widget fail your corporate WCAG compliance audits.

Exercise 4.1: Cross-Platform Journey Usability Mapping

Task: Design usability test cases to verify that session context is preserved during transitions and that returning to the portal preserves user state instead of throwing them back to a generic dashboard.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-4.1.1	Employee Portal ↔ embedded HR Benefits iFrame - SSO session handshake and back-navigation	Log into Portal, click the Benefits link, perform an action inside the iframe (e.g., view a pay stub), then click 'Return to Portal'	1) Confirm the SSO handshake completes silently (no visible re-login prompt) when the iframe loads. 2) Perform a stateful action inside the iframe (e.g., open a specific benefits document). 3) Click 'Return to Portal' or use the browser back button. 4) Assert the Portal restores the user's prior context (same tab/section/filter) rather than resetting to the default dashboard. 5) Assert no credential re-entry is required at any point in the round trip.	PASS: Session and UI state are fully preserved across the transition with no re-authentication. FAIL: User is forced to log in again, or is dropped onto the generic default dashboard, losing prior context.

Exercise 4.2: WCAG Accessibility Audit for Embedded Widgets

Task: Design audit checklists to verify the third-party widget satisfies four critical accessibility criteria: 1) Keyboard Navigation, 2) Screen Reader Labels, 3) Focus Management (no focus traps), and 4) Color Contrast.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-4.2.1	Embedded HR Benefits iframe widget - keyboard operability	Full keyboard-only pass (Tab, Shift+Tab, Enter, Space, Esc) across every interactive element, no mouse used	1) Tab through the widget from top to bottom and assert every interactive element (links, buttons, form fields) receives visible focus in a logical order. 2) Assert Enter/Space activates buttons and links as expected. 3) Assert Esc closes any open menus/modals. 4) Assert focus never becomes invisible or unreachable (no element skipped). 5) Assert there is no point where keyboard focus becomes trapped unintentionally.	PASS: 100% of interactive elements are operable via keyboard alone, in logical order. FAIL: Any control is unreachable, unresponsive to keyboard input, or focus becomes trapped/lost.
TC-4.2.2	Embedded widget form fields and buttons - screen reader accessibility (NVDA/VoiceOver)	Run a screen reader pass over all form inputs, buttons, and dynamic status messages in the widget	1) Turn on a screen reader (NVDA or VoiceOver) and navigate the widget. 2) Assert every form input has an associated, meaningful label (via <label> or aria-label) - not just a placeholder. 3) Assert every button announces its actual function (e.g., 'Submit claim', not just 'button').	PASS: All elements have correct, descriptive accessible names and dynamic content is announced. FAIL: Any input, button, or status update is unlabeled, mislabeled, or silent to the screen reader.

			4) Assert dynamic updates (e.g., 'Claim submitted') are announced via an aria-live region without requiring the user to re-navigate. 5) Assert image/icon-only controls have appropriate alt text or aria-label.	
TC-4.2.3	Modal dialogs rendered inside the embedded Benefits iframe - focus management	Open an in-widget modal (e.g., 'Confirm claim submission'), then attempt to Tab out of it	1) Trigger the modal to open and assert focus moves into the modal automatically. 2) Tab repeatedly and assert focus cycles only within the modal's interactive elements while it is open (intentional trap). 3) Press Esc or click the close control and assert the modal closes. 4) Assert focus returns to the exact element that triggered the modal. 5) Confirm no other part of the page (outside intentional modals) ever traps keyboard focus.	PASS: Focus is correctly trapped only inside open modals and restored to the trigger element on close. FAIL: Focus escapes the modal while open, or becomes permanently stuck/lost after closing.
TC-4.2.4	All text and interactive components within the embedded widget - color contrast	Automated scan (axe-core / Lighthouse) across every screen/state of the embedded widget	1) Run an automated accessibility scanner across all widget screens, including hover/focus/error states.	PASS: Zero contrast violations found across all scanned states, meeting WCAG 2.2 AA. FAIL: Any text or UI

			2) Assert normal body text meets a contrast ratio of at least 4.5:1 against its background. 3) Assert large text (≥18pt or 14pt bold) meets at least 3:1. 4) Assert non-text UI components (buttons, form borders, focus indicators) meet at least 3:1 against adjacent colors. 5) Manually spot-check any scanner warnings that require human judgment.	component falls below its required contrast ratio.

Exercise 4.3: Federated SSO Usability & Redirect Loops

Task: Write a test case evaluating OAuth/SAML redirect loops, ensuring credentials are never exposed in URLs during the session handshake.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-4.3.1	OAuth2/SAML federated SSO handshake between the Portal and the Identity Provider (IdP)	Intercepted network traffic (proxy/dev tools) during login; also force an expired/invalid SAML assertion to trigger a failure path	1) Initiate login and capture every redirect in the handshake chain using a network proxy. 2) Assert no access token, ID token, or SAML assertion ever appears in a URL query string - only in POST bodies, headers, or fragments per spec. 3) Force an invalid/expired	PASS: Credentials/tokens never appear in URLs, and failures resolve to a single error page with no loop. FAIL: A token/assertion is exposed in a URL, or the user is caught in a repeating redirect loop.

			assertion and assert the flow redirects to a single, clear error page (not back into the login flow). 4) Attempt to retry login after the failure and assert the system does not re-enter an infinite redirect loop (max 1 retry redirect). 5) Confirm all redirect URIs are on an allow-list (no open-redirect vulnerability).	

Section 5: Metric Gates & Flaky Test Quarantine

Background Scenario: Integration tests can be flaky—failing intermittently (e.g., 5% of the time) due to shared test environment latency, transient network blips, or external API downtime. If flaky tests are ignored in the main deployment pipeline, developers begin ignoring all automated test failures, destabilizing the product. Managing quality at scale requires strict automated gates and the implementation of a clear Quarantine Standard.

Exercise 5.1: Flaky Test Remediation & Quarantine Strategy

Task: Design a multi-tiered remediation plan to handle a flaky Checkout API test. Explain how the test will be safely quarantined and what mock or service virtualization mechanisms will be established to stabilize it while maintaining overall pipeline trust.

Test Case ID	Technical Target (Endpoint/Schema)	Test Input (JSON / Variable)	Assertion / Verification Steps	Pass/Fail Gate Criteria
TC-5.1.1	Checkout API integration test suite - flaky test `test_checkout_completes_payment` (~5% intermittent failure rate)	CI history of 100 runs showing ~5 intermittent failures tied to third-party payment gateway network variability	1) Confirm flakiness statistically (re-run the test 20–30x in isolation) to rule out a genuine intermittent bug. 2) Tag the test with a @quarantine marker so it runs in CI but does not block merges. 3) Route the quarantined test to a separate, non-blocking CI job, while the main pipeline gate ignores its result. 4) Replace the live third-party payment gateway call with a service-virtualization mock (e.g., WireMock/Mountebank) to eliminate the external network variable. 5) Track the quarantined test on a dashboard with a fix SLA (e.g., 2 weeks) and	PASS: Test is safely quarantined without destabilizing the pipeline, root cause is mitigated via virtualization, and it is re-promoted after 20 consecutive passes. FAIL: Test is left blocking the pipeline (eroding trust) or is silently deleted without root-cause remediation.

			an owner. 6) Re-promote the test to the blocking suite only after 20 consecutive green runs post-fix.	

Exercise 5.2: CI/CD Automated Quality Gate Scorecard

Task: Complete the automated pipeline gate scorecard below. Define the industry standard tool, the passing metric threshold, and the remediation action on failure for each quality phase.

Pipeline Stage	Industry Standard Tool	Metric Threshold to Pass	Remediation Actions on Failure
Stage 1: Static Code Analysis	SonarQube (static analysis) / ESLint or equivalent linter	0 Blocker/Critical issues; ≥80% coverage on new code; 0 high-severity security hotspots	Block the merge automatically; auto-assign a ticket to the code owner; require a fix or a documented, approved suppression before re-running the gate
Stage 2: Unit Testing	Jest / JUnit / PyTest (per stack)	100% pass rate on the blocking unit suite; ≥80% line coverage	Fail the build immediately; notify the commit author via CI bot comment; block merge until all unit tests pass
Stage 3: API Contract Testing	Postman/Newman or Pact (consumer-driven contracts) validated against the OpenAPI spec	100% schema compliance; 0 breaking changes detected against existing consumer contracts	Block deployment; auto-generate a contract-diff report; require explicit provider/consumer sign-off before the gate reopens
Stage 4: Dynamic Application Security (DAST)	OWASP ZAP (or Burp Suite Enterprise) automated DAST scan	0 High/Critical findings (OWASP API Top 10); Medium findings capped at an agreed threshold (e.g., ≤3)	Block the production release; file a security ticket with a severity-based SLA (Critical = 24h); mandatory re-scan before the release gate reopens